

Поиск черной кошки в 2000-мерной темной комнате. Турнир алгоритмов машинного обучения

Эпиграф:

“Эксперимент завершен. Результаты выглядят так, будто я немного сломал законы физики привычного табличного ML”

“Может быть это соревнование, которого не должно было быть?”

“Это приглашение к репликации”

Спрятать иголку в стоге сена? Да!

Добро пожаловать на мой маленький тестовый полигон.

В этой статье я расскажу, как столкнулся лбами двадцать один алгоритм машинного обучения - от старой доброй линейной регрессии, KNN, случайного леса до троицы табличных королей (XGBoost, LightGBM, CatBoost), нескольких многослойных нейросетей и нейросетей с механизмом внимания. И я заставил их всех решать задачу, которая на первый взгляд кажется абсурдной (только на первый взгляд?).

Большинство бенчмарков в машинном обучении вроде MNIST или Titanic давно натренированы до дыр. На картинках побеждают свертки, а на табличных данных – градиентный бустинг. Предсказуемо. Скучно.

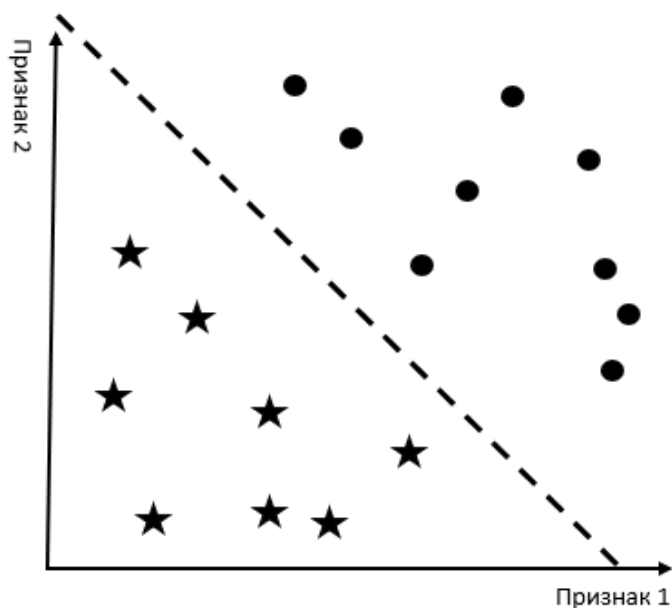
Поэтому я решил устроить особый стресс-тест, соревнование немного иного формата, проверка на предельную прочность на табличных данных.

Да... еще среди участников забега будет один новичок, о котором большинство читателей, вероятно, слышит впервые. Он не раскручен и не имеет армии поклонников на Kaggle. Но уже имеет красивое название – Полигармонический каскад. Это глубокая архитектура, выведенная из принципов теории случайных функций и индифферентности. В этом тестировании он выступал в роли новичка-аутсайдера. Но то, что он сделал с фаворитами, выглядит как читерство. Но об этом позже.

Итак, что же это за задача?

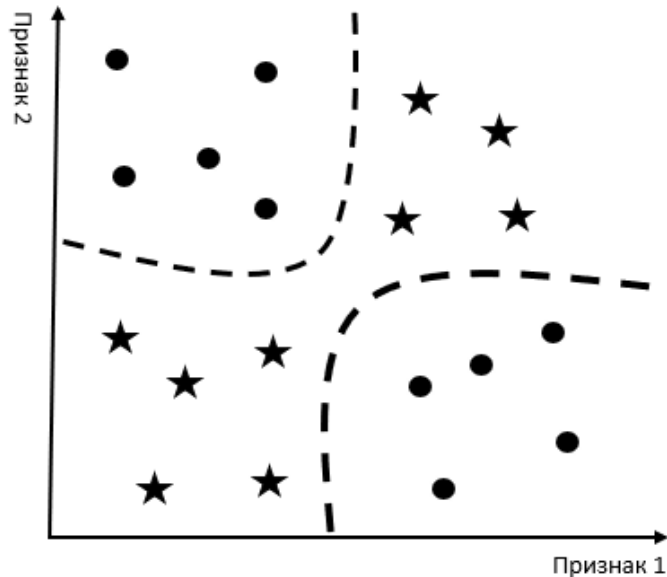
Попробую сформулировать сначала первую идею

Обычно, когда говорят об основах машинного обучения, в качестве самого первого примера, рисуют что-то подобное:

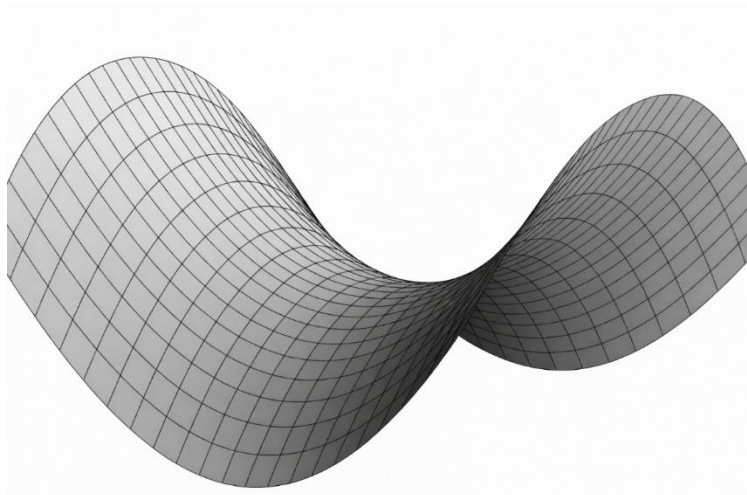


Но это слишком просто.

Далее обычно рисуют что-то чуть сложнее:



Это тоже слишком просто. Тут классы легко разделяются простейшей нелинейной поверхностью. Например, какой-то такой:



Похожей на гиперболический параболоид.

Если сразу так и сформулировать, поставить цель – получить эту поверхность, функцию, тогда это уже будет регрессионная задача.

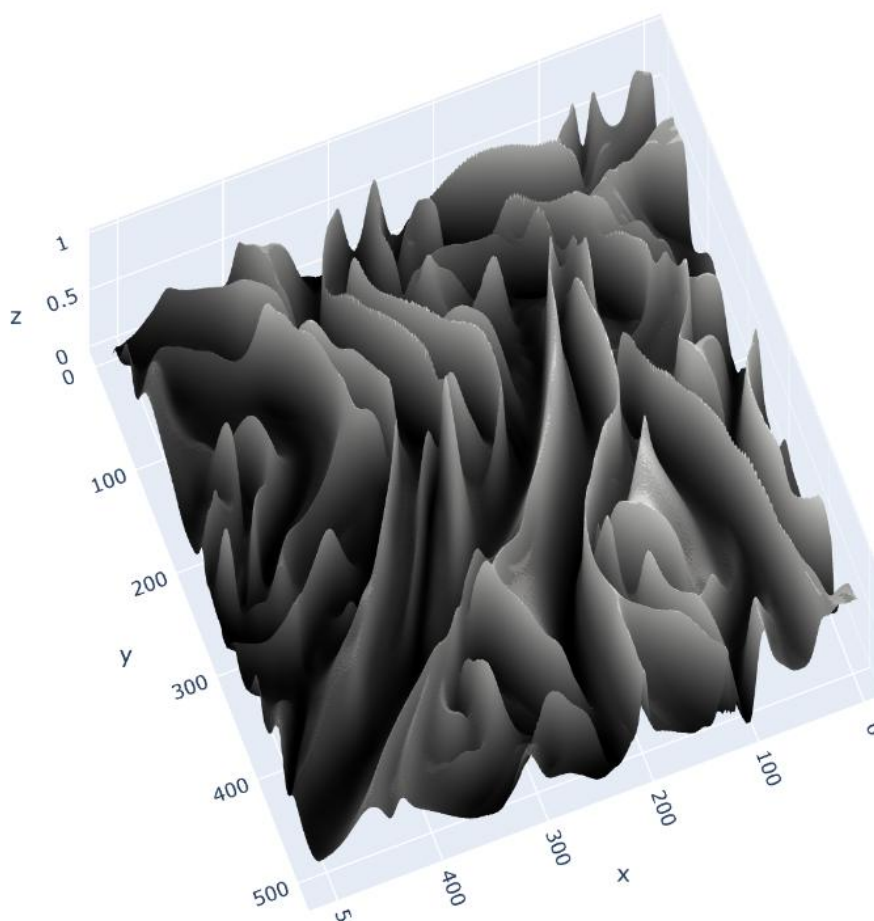
Чем более нелинейной будет функция, чем больше у неё локальных минимумов и максимумов, тем сложнее обучение. Логично.

Как на счет функции заданной вот таким изображением? Красиво?



Берем это изображение и превращаем его в регрессионную задачу. Два входных признака – координаты x и y . На выходе нормализованная яркость от 0 до 1. Никакой классификации, никаких котиков и собак. Просто по точке на плоскости скажи, насколько там светло.

Если превратить это в 3D поверхность, получится вот что:



Это уже чуть сложнее, чем гиперболический параболоид. Правда? Пусть ML-методы попробуют этому обучиться!

Но вы ведь скажите, что всё равно это обычная интерполяция и слишком просто? Не торопитесь, это было лишь вступление.

Разрешение 512 на 512 пикселей даст выборку в 262144 примеров, что можно разделить: 240000 на обучающую выборку и 22144 примера на тестовую. Предварительно перетасуем их случайным образом конечно же. Вполне неплохо для тестов.

Но повторюсь, вы ведь наверняка подумали, как и я тоже, что эта задача все равно кажется слишком простой для современных методов! Звучит... ну как обычная интерполяция, хоть и очень сложной двумерной функции.

В чем подвох?

Но я не был бы собой, если бы не превратил эту задачу в маленький кошмар. Я взял и искусственно увеличил размерность входного пространства. Вместо двух признаков x и y подадим на вход 500, 1000, а затем и 2000 признаков. Из них только два будут настоящие координаты (но какие именно алгоритмам будет неизвестно). Остальные 498, 998 и 1998 признаков пусть будет шум.

		Информативный признак x																	Информативный признак y								
		1000 признаков (шум)																									
Обучающая выборка	r1	r2	r3	r4	r5	r6	x	r8	r9	r10	r11	r12	r13	r14	r15	r16	r17	r18	y	r20	r21	r22	...				
	58	50	2	35	25	51	34	13	82	39	1	20	81	46	9	82	61	40	29	71	23	63	39				
	42	76	94	98	54	84	54	9	5	66	85	52	52	61	76	36	6	77	55	7	8	27	88				
	1	83	50	46	27	70	41	65	22	38	37	6	51	12	16	69	56	76	13	66	10	65	4				
	55	26	32	14	63	1	22	14	64	82	23	52	70	61	81	42	62	14	27	55	96	13	95				
	79	7	4	27	74	65	92	21	38	43	17	76	17	38	78	80	40	32	50	30	16	76	48				
	53	15	92	64	40	52	64	31	14	91	9	10	46	24	88	43	9	22	14	91	23	37	41				
	48	18	3	62	30	84	83	30	33	41	76	17	30	72	74	21	70	40	27	52	25	83	25				
	79	12	55	35	55	20	30	36	87	24	87	10	27	92	58	71	51	96	51	55	14	47	56				
	31	95	96	87	31	43	7	25	57	10	73	29	19	99	84	4	38	29	70	99	40	47	2				
	15	85	0	79	67	23	79	78	43	8	30	74	89	52	32	25	78	97	60	62	52	94	17				
	39	100	86	69	10	44	61	91	96	43	60	55	73	83	62	60	63	82	14	76	96	100	96				
	50	10	59	72	42	86	98	31	46	60	48	75	90	72	44	31	52	100	73	18	52	59	44				
	79	74	55	48	26	1	26	81	58	96	86	28	67	48	42	73	48	47	66	33	75	55	13				
	66	10	80	87	74	83	71	68	28	99	92	69	73	11	29	3	25	34	20	47	29	70	25				
	87	52	91	3	68	72	29	85	27	0	49	95	94	23	51	84	19	40	64	70	66	79	60				
	9	61	62	1	76	98	75	8	61	83	42	11	73	93	39	73	67	48	29	37	13	85	23				
	35	39	35	48	72	91	19	43	2	81	84	6	84	95	6	66	15	57	79	38	12	22	20				
	54	18	97	52	3	36	33	21	59	65	24	39	2	98	84	36	80	9	73	48	84	14	75				
	8	55	42	72	36	95	81	48	31	54	13	23	91	52	56	20	27	12	33	44	7	53	60				
	98	76	54	47	84	25	17	41	91	23	44	91	28	53	80	36	25	33	62	30	14	13	12				
	8	14	27	27	39	72	92	30	12	82	89	93	82	64	94	69	31	25	34	22	14	62	94				
	18	34	90	31	22	28	62	19	31	84	40	44	40	48	52	41	86	79	26	7	13	74	13				
	98	4	87	31	69	50	94	18	57	95	73	51	17	88	97	25	56	47	66	100	51	88	49				
	27	57	49	3	80	10	85	54	27	34	42	12	44	9	28	94	57	73	85	55	32	48	51				
	43	72	38	53	86	53	58	3	53	31	10	6	29	60	24	78	66	34	71	13	82	62	14				
	39	5	7	66	86	50	37	46	88	92	71	50	68	94	23	60	30	28	35	13	82	95	97				
	5	82	23	88	46	45	80	30	86	64	11	4	61	92	71	87	41	71	58	9	59	5	67				
	11	1	18	64	14	96	20	7	79	54	84	57	31	73	83	73	3	64	49	40	58	72	40				
	20	52	35	97	56	44	1	8	15	20	62	39	65	33	19	33	58	59	65	25	66	71	30				
	70	77	33	31	87	91	44	35	96	77	5	16	53	65	7	26	97	74	99	55	45	54	39				
	87	1	35	25	57	8	90	36	52	6	96	83	27	30	61	89	14	14	45	45	66	61	5				

Но этот шум не простой. Каждый шумовой признак, это случайно перетасованные значения всё тех же x и y. Сами по себе они не отличаются от полезных сигналов: то же распределение, та же амплитуда, те же средние. Отличить настоящие координаты от фальшивых просто глядя на входную часть данных невозможно.

Это классическая проблема отбора признаков, но доведенная до предела. Алгоритм должен не просто выучить очень сложную функцию, но еще и понять, от каких именно двух переменных из тысяч, это нужно сделать.

Но это классическая ситуация в реальном мире. Финансовые данные, геномные последовательности, показания сотен датчиков. Не известно заранее, какие из измерений важны, а какие мусор.

А теперь неожиданный поворот. Матричный поворот!

Ну что же... Задача теперь стала заметно сложнее и интереснее. Но хочется же добавить в нее что-то ещё! Ещё одну дополнительную опцию, с которой она станет еще более красивой и интересной!

В базовом варианте два информативных признака это просто две колонки в таблице данных (хотя и неизвестно какие).

Многие алгоритмы, особенно древовидные, умеют находить полезные признаки перебором: “А что, если я разобью данные по признаку номер 5? А если по признаку номер 117?”. Это работает, когда информативные оси совпадают с осями координат.

Но что, если я сделаю так: возьму всё это многомерное пространство признаков (и информативных, и шумовых) и поверну его на случайный угол? Умножу на случайную матрицу поворота? Не меняя расстояние между точками, не меняя геометрию. Просто ортогональное преобразование.

После такого поворота ни один отдельный признак не несёт полной информации. Полезный сигнал оказывается размазан тонким слоем по всем двум тысячам координат. Если раньше полезный сигнал лежал на условном “полу” (по осям X и Y). После поворота этот “пол” перевернулся и завис где-то под странным углом в 2000-мерном космосе.

Для человека или алгоритма, анализирующего столбцы по-отдельности, задача стала абсолютно неразрешимой.

Как измерим успех или провал?

Каждый алгоритм после обучения получает на вход новую тестовую выборку в 262144 примеров. Но с одной важной деталью: шумовые признаки в тестовых данных новые, сгенерированные заново случайные числа (снова путем перестановки, но другим способом). Если алгоритм просто запомнил обучающую выборку, на тесте он провалится мгновенно.

Результат измеряется одной цифрой - среднеквадратичной ошибкой (RMSE) относительно эталонного изображения. $RMSE = 0$ означает идеальную работу. $RMSE \sim 0.2$, как мы увидим, это уровень «я сдаюсь, вот вам средняя яркость».

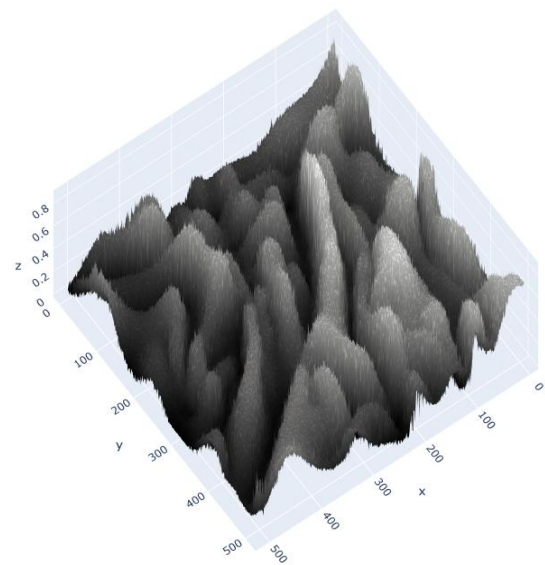
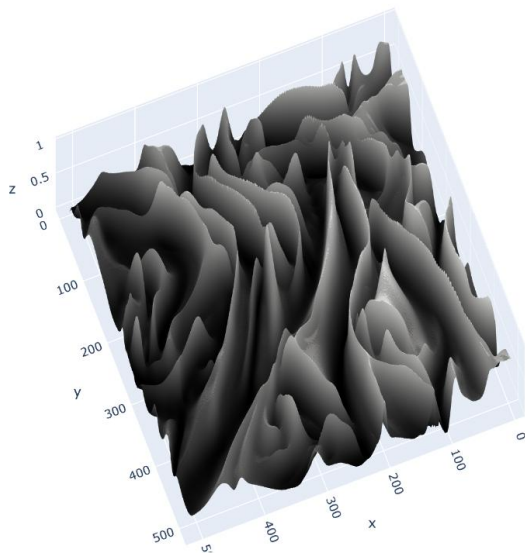
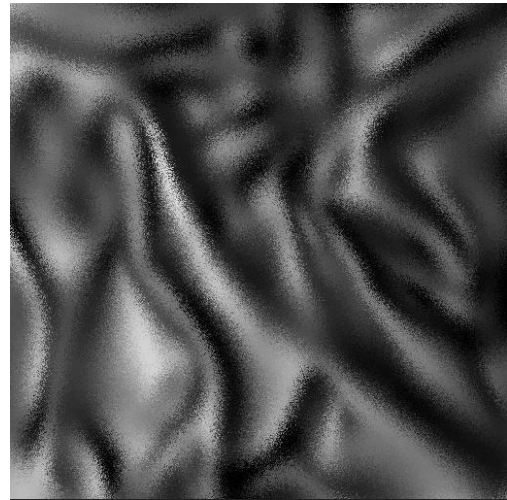
Кроме того, на этапе окончательного тестирования обученной модели можно упорядочить такую тестовую выборку так же, как последовательно расположены пиксели на изображении. Тогда результат ее обработки моделью можно легко превратить в новое изображение, которое просто визуально и наглядно можно сравнить с исходным. Либо можно построить и наглядно сравнить между собой две 3d поверхности.

Например, что-то такое:

эталон



результат



Итак, давайте посмотрим, что же произошло, когда я бросил вызов современным методам.

Кто же вышел на старт?

У любого уважающего себя соревнования должен быть список участников. У меня их набралось два десятка, если считать с вариациями настроек.

Участники:

- Кандидат 1: Ridge регрессия
- Кандидат 2: kNN
- Кандидат 3: SVR

- Кандидат 4: HistGradientBoostingRegressor (настройки Claude)
- Кандидат 5: HistGradientBoostingRegressor (настройки по умолчанию)
- Кандидат 6: LightGBM (настройки Claude)
- Кандидат 7: LightGBM (настройки по умолчанию)
- Кандидат 8: XGBoost (настройки Claude)
- Кандидат 9: XGBoost (настройки по умолчанию)
- Кандидат 10: CatBoost (настройки Claude)
- Кандидат 11: RandomForest (настройки Claude)
- Кандидат 12: RandomForest (настройки по умолчанию)
- Кандидат 13: ExtraTrees (настройки Claude)
- Кандидат 14: ExtraTrees (настройки по умолчанию)
- Кандидат 15: Нейросеть 1 (bottleneck, код Claude)
- Кандидат 16: Нейросеть 2 (два режима, код Claude)
- Кандидат 17: Нейросеть 3 (собственная моя реализация)
- Кандидат 18: TabNet вариант 1 (настройки Claude)
- Кандидат 19: TabNet вариант 2 (настройки Claude)
- Кандидат 20: TabNet вариант 3 (настройки по умолчанию)
- Кандидат 21: Полигармонический каскад (моя реализация)

И вот тут я должен описать одно методологическое решение.

Я прекрасно понимаю, как работают ученые. Если кто-то придет и скажет: “Вот этот алгоритм у меня порвал остальных конкурентов в клочья”, первая реакция может быть “Ты просто криво реализовал остальные алгоритмы или подобрал гиперпараметры так, чтобы на твоей задаче они сломались”. Это справедливая паранойя.

Поэтому я решил подстраховать свою репутацию максимально циничным способом. Я не стал писать код для широко известных методов сам.

Я подробно описал всю суть соревнования, формат данных, шумовые признаки, хитрость с матричным поворотом, критерии успеха, предоставил код генерации выборок, а затем отдал всё на откуп Claude Opus 4.6.

Попросил топовую (на тот момент) нейросеть написать тесты, сделать адаптивные настройки гиперпараметров и вообще сделать всё, чтобы бустинги и нейросети показали на этой задаче свой абсолютный лучший результат.

Клод справился: код получился аккуратным, с ранними остановками, с тонкой настройкой под разные размерности. Одед существующие методы в лучшую (по его мнению) броню.

Впрочем, на этапе тестов у меня возникла вторая мысль: а вдруг Клод, будучи чрезмерно старательным, где-то мог перемудрить (например, слишком выкрутил какие-то гиперпараметры из страха перед тысячами шумовых признаков)? Поэтому для части методов были мною созданы дублирующие скрипты. Абсолютно голые методы, с настройками по умолчанию из коробки. Пусть выживает тот, кто выживает по-настоящему.

Ну и, наконец, чтобы не терять связь с реальностью, один из вариантов нейросетей я написал полностью самостоятельно, на чистом PyTorch, как это делали раньше обычные люди.

Теперь, к участникам. Если хотите, давайте пройдемся по ним кратко по группам. Или можете сразу (если они Вам все знакомы) пролистнуть эту часть и перейти к результатам.

Если сгруппировать методы по тактике ведения боя, мы получим следующую картину:

Линейная классика.

Ridge-регрессия - это метод, который пытается провести прямую через точки. Или плоскость. Или гиперплоскость (смотря сколько измерений). В любом случае он бессилен против моей извилистой поверхности.

Но я включил его намеренно: пусть он будет как эталон бессилия! Кто-то же должен взять на себя эту роль?

Если какой-то метод показывает такой же RMSE, как линейная регрессия, значит, считаем, что он не выучил вообще ничего.

Обозначение на диаграммах — **Ridge**.

Метод ближайших соседей (k-NN).

Этот парень обладает фотографической памятью: запоминает все обучающие примеры, а для нового объекта ищет ближайших соседей. Но добавьте шумовые признаки и понятие «ближайший» теряет смысл. Посмотрим, как он справится с проклятием размерности.

Обозначение на диаграммах – **kNN**.

Метод опорных векторов (SVR).

Классический kernel-метод. Красив математически, но на больших выборках его кубическая сложность по числу примеров превращает обучение в медитацию. Пришлось использовать подвыборку в 10 тысяч точек.

Обозначение на диаграммах – **SVR(10k)**.

Древесная аристократия.

Случайный лес и его экстремальный аналог ExtraTrees. Оба строят ансамбли деревьев. Старая школа, надежная. Они способны перебирать координатные оси. Посмотрим, как далеко это их заведёт. Для каждого из них две версии: одна с тонкой настройкой от Клода, другая «из коробки».

Обозначение на диаграммах – **RF(cl)**, **ExtraT(cl)**, **RF(def)**, **ExtraT(def)**.

Бустинг-тяжеловесы.

Здесь собралась элита табличных соревнований.

HistGradientBoosting – быстрый и эффективный, вдохновлённый LightGBM.

Сам LightGBM – мастер гистограммных трюков и работы с GPU.

XGBoost – ветеран, прошедший сотни Kaggle-баталий.

CatBoost – продукт Яндекса, славящийся умением обрабатывать категориальные признаки (которых у нас нет, но он всё равно хороший).

Каждый из них получил по две версии: настроенную Клодом и заводскую (кроме CatBoost, для которого заводские настройки оказались хуже и поэтому в соревнование не вошли). Все они умеют находить важные признаки... пока эти признаки остаются отдельными колонками в таблице.

Обозначение на диаграммах – **HistGB(cl)**, **HistGB(def)**, **LGBM(cl)**, **LGBM(def)**, **XGB(cl)**, **XGB(def)**, **CatBoost**.

Нейросети.

Три разных архитектуры (на PyTorch, оптимизатор Adam). Первые две – результат творчества Клода: многослойные сети с узким горлом, призванным заставить модель выделить самую суть. Третья – моя собственная реализация, более простая архитектурно. Нейросети теоретически точно способны выучить поворот координат. Вопрос в том, справится ли оптимизационный алгоритм и хватит ли им на это терпения и вычислительных ресурсов.

Обозначение на диаграммах – **MLP-1, MLP-2, MLP-3.**

TabNet.

Экзотический гость. Нейросеть, специально спроектированная для табличных данных, с механизмом внимания, который должен сам отбирать важные признаки. Три версии с разными настройками. Посмотрим, оправдает ли внимание возложенные надежды.

Обозначение на диаграммах – **TabNet-1, TabNet-2, TabNet-def.**

Аномалия. Полигармонический каскад.

Полигармонический каскад в этом хороводе выделяется отдельно. Это не нейросеть. Его обучение устроено иначе, чем стандартный градиентный спуск. Это участник, пришедший на соревнование со своим, слегка подозрительным тренировочным режимом. Весь его код написан мной лично, в соответствии с недавним циклом научных статей (декабрь 2025), ссылки на которые (на ArXiv и preprints.ru) будут в конце. Чистая, нетронутая математика.

Обозначение на диаграммах – **PHC Ne** (где N – количество эпох). В каждом тесте полигармонический каскад (чтобы лучше понять зависимость его качества от времени обучения) испытывался несколько раз, с разным количеством эпох.

Код всех тестов лежит в открытом доступе на GitHub (ссылка будет в конце).

Вы можете проверить, воспроизвести, найти ошибки или улучшить. Я только «за».

А теперь, когда все участники вооружены, а правила честности соблюдены (даже за счет привлечения постороннего искусственного интеллекта), давайте посмотрим, к чему всё это привело.

Мой тестовый полигон основан на процессоре Intel® Core™ i9-10920X и видеокарте NVIDIA GeForce RTX 3070 8 Гб. Для тех алгоритмов, где возможно и полезно, обучение было на GPU. Может быть моё техническое вооружение не идеально и для других конфигураций могли получиться иные результаты (в первую очередь по времени).

Старт соревнования: разминка и первый удар под дых

Как и в любом достойном соревновании, первые раунды были разминочными. Но и они важны: на них быстро видно, кто вообще понимает правила игры. Чистая проверка на вменяемость.

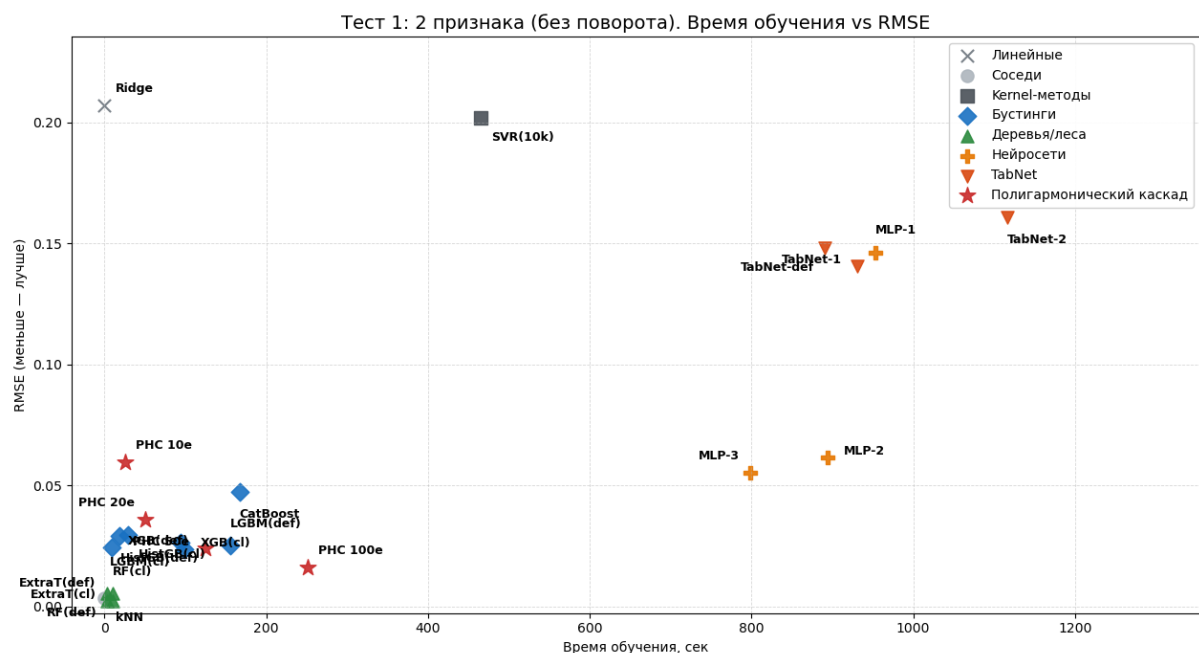
Три разминочных раунда:

1. Раунд 1. Просто 2 честных признака на входе, никакого шума.
2. Раунд 2. 10 признаков: 2 полезных + 8 шумовых “двойников”.
3. Раунд 3. Те же 10 признаков + случайный поворот признакового пространства.

Оговорка: при всего двух честных признаках на входе делать поворот не имеет смысла. Это будет просто поворот самой плоскости. Два признака останутся информативными как были. Поэтому поворот делаем только уже при добавлении шума.

Раунд 1. Двумерный спринт.

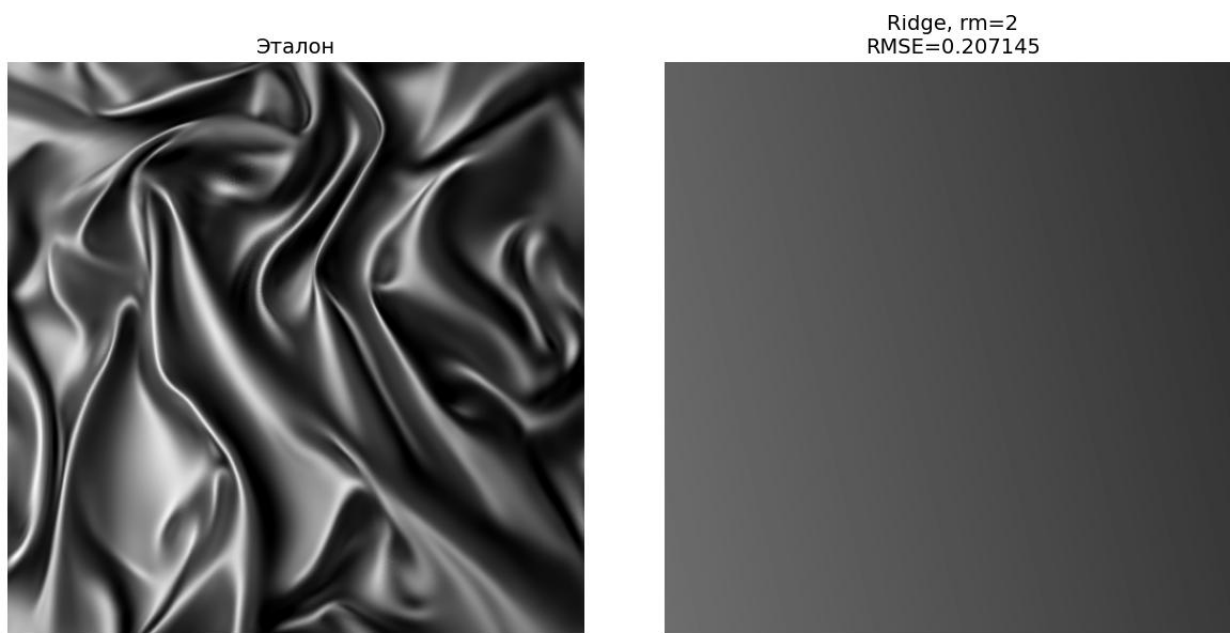
Когда на входе только x и y , задача превращается в обычную двумерную интерполяцию 3D-поверхности. Большинство участников справились.



Разберем первый раунд чуть более подробно, чтобы заодно показать, что означают разные значения RMSE фактически.

Выбыли из соревнований двое.

Линейная регрессия (Ridge) ожидаемо сдалась сразу. Но покинула соревнование с гордо поднятой головой, выполнив важную роль эталона беспомощности, получив RMSE 0.207 (важное значения, для сравнения в дальнейшем).

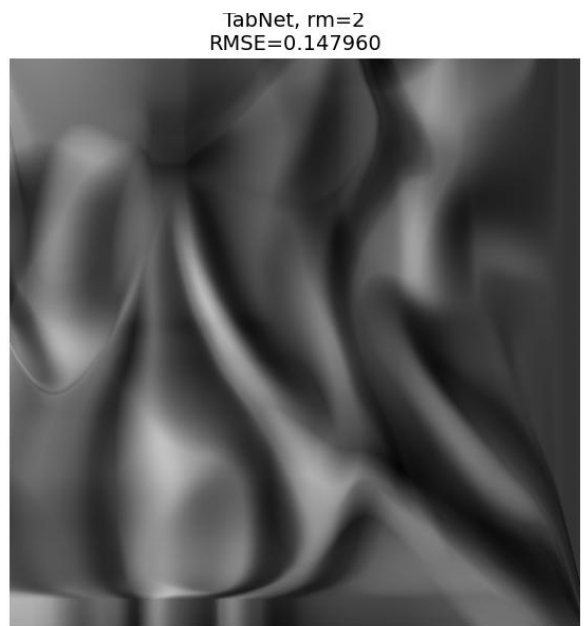
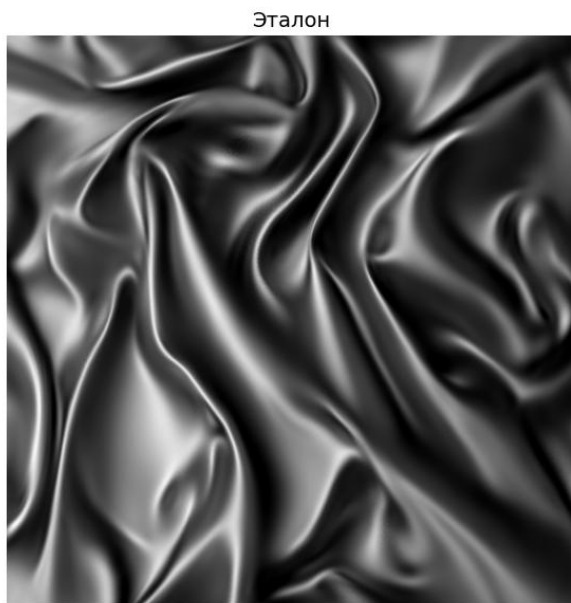


Как видно по “восстановленному” изображению, значение RMSE выше 0.2 означает, что функция фактически не была воспроизведена никаким образом.

Метод опорных векторов SVR(10k), даже на урезанной выборке, тоже не справился (0.202), потратил почти 8 минут и отправился отдыхать (возможно, можно подобрать для него параметры лучше, но в этом турнире это был контрольный выстрел).

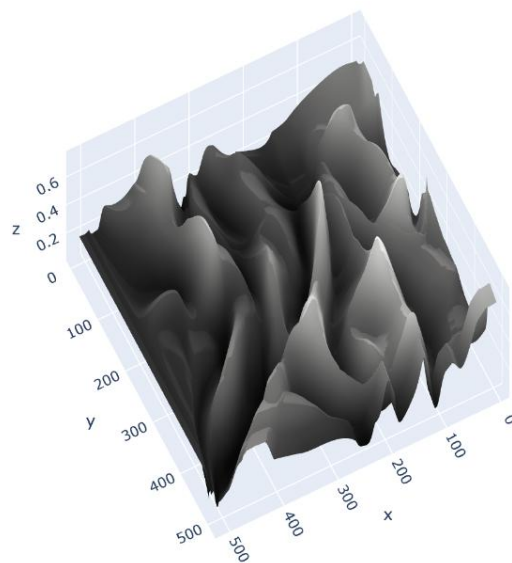
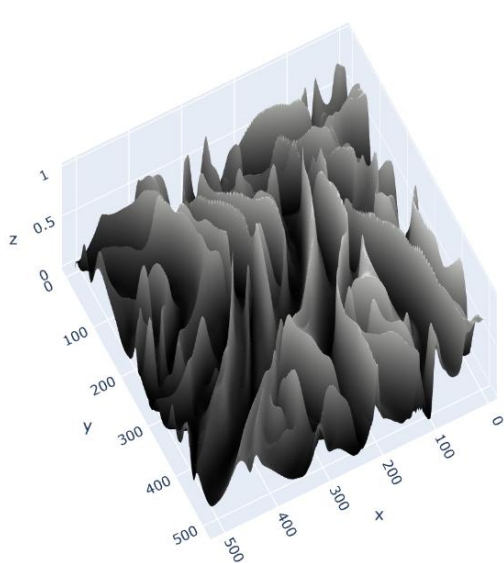
Нейросети обучались в пределах 13 – 19 минут. Градиентному спуску нужно время, чтобы “пробиться” к сложной поверхности? В итоге нейросети условно разделились на две группы. MLP2 и MLP3 показали значения RMSE 0.0614 и 0.055. Остальные (MLP-1 и все TabNet) показали значительно более слабый результат, застряв в диапазоне 0.14 – 0.16. Механизм внимания, видимо, не понял на что обращать внимание, когда выбор тривиален.

Визуализируем для лучшего понимания в дальнейшем, что такие диапазоны значений RMSE означают.



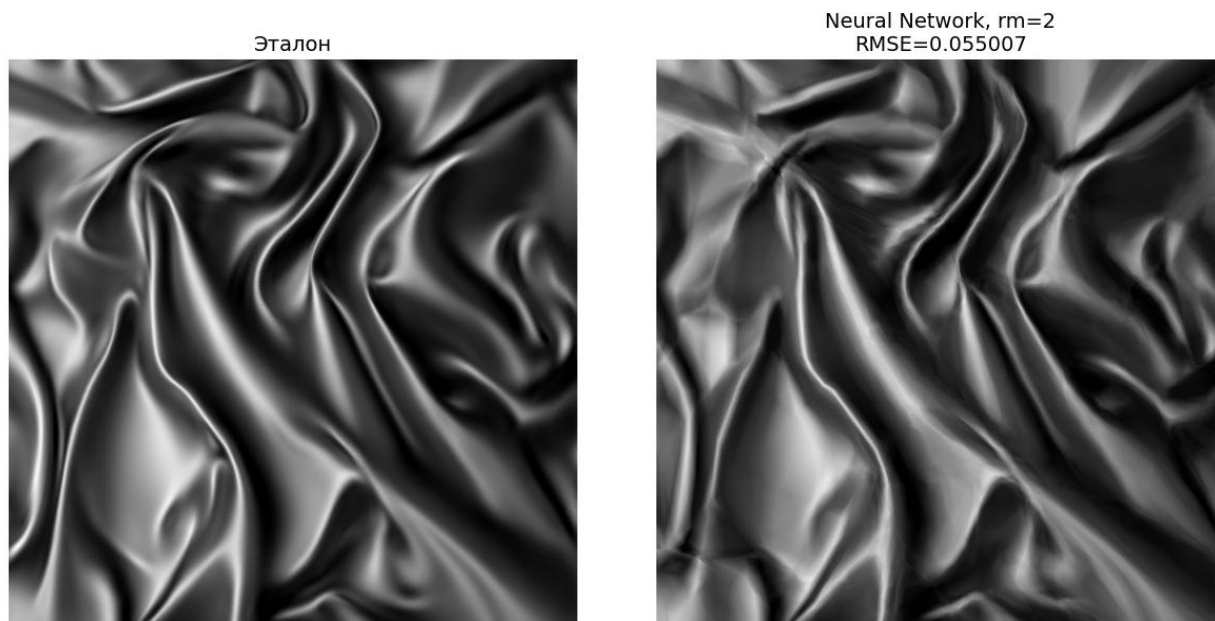
Пример Tabnet-def с RMSE 0.14796. Этому значению RMSE соответствует выделение наиболее крупных деталей, холмов и впадин, но отсутствие каких-либо мелких деталей.

Если в виде поверхности:



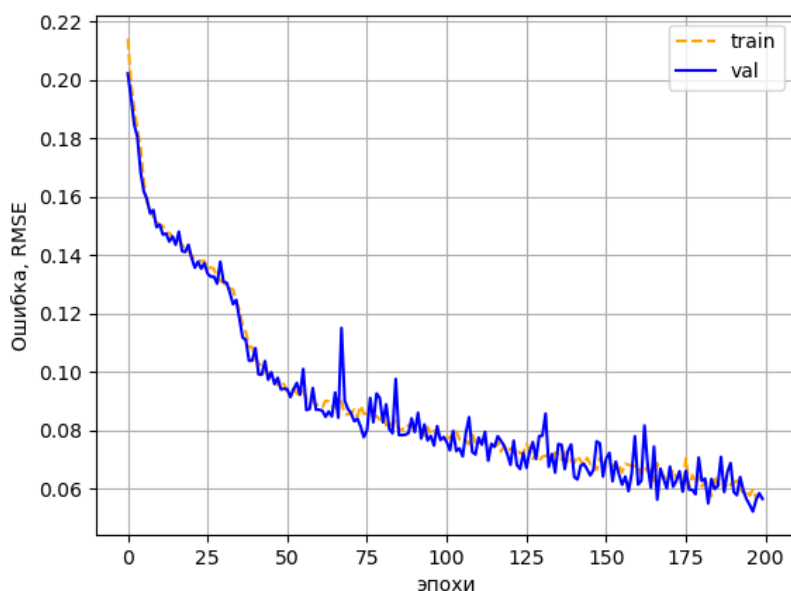
Эталон и Tabnet-def с RMSE 0.14796

Теперь посмотрим для сравнения, на что похоже MLP-3 с результатом 0.055.



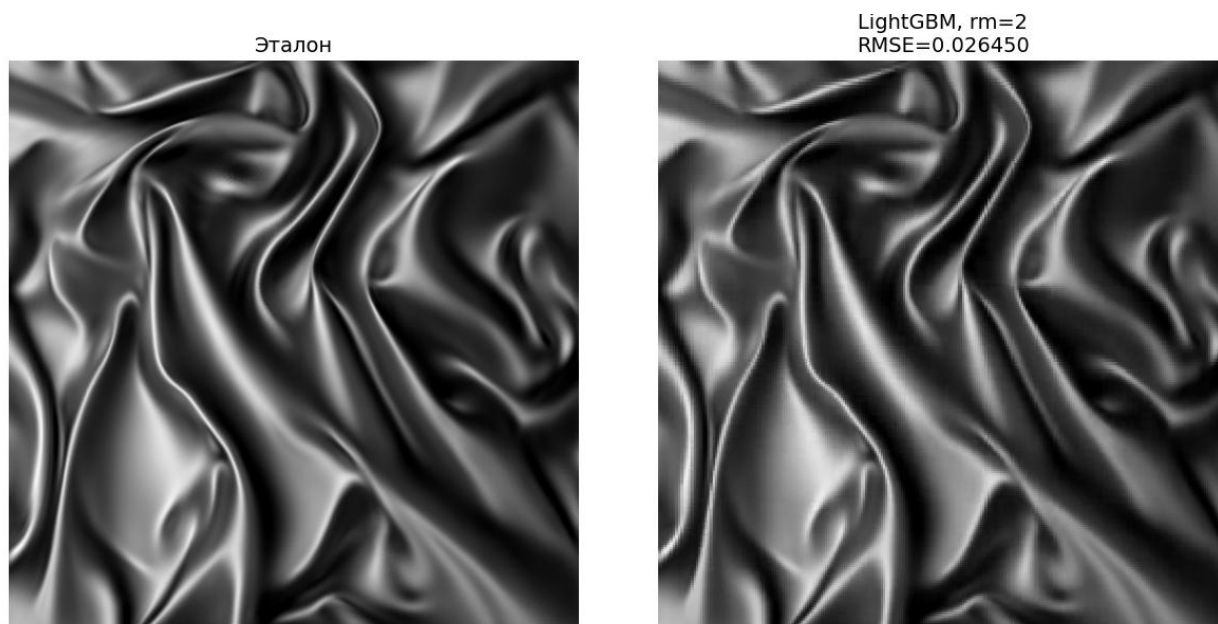
Видно, что значение RMSE 0.055 означает, что нейросеть воспроизвела практически все основные нелинейности функции, разойдясь с эталоном в мелких деталях. Т.е. RMSE в диапазоне 0.055 это уже вполне неплохо.

Если посмотрим на кривую обучения:



Видно, что нейросеть могла продолжить обучение и дальше, улучшая результат, но специально была мной ограничена по количеству эпох, чтобы не слишком выделяться по времени в этом тесте с другими методами (тем более, что результат уже достаточный, чтобы пройти в следующие раунды).

Бустинги LightGBM, XGBoost, HistGradientBoosting взяли барьер в первом раунде с результатами около 0.023–0.029. Лучше, чем нейросети, да и еще намного быстрее.



Результат работы LightGBM (с настройкой от Клода) с RMSE 0.02645. Как видно, если RMSE меньше 0.03, это значение соответствует очень высокому качеству и на первый взгляд полученную функцию уже сложно различить с эталоном, если не вглядываться в самые мелкие детали.

CatBoost немного отстал (0.047) от других бустингов, но результат для первого раунда достойный.

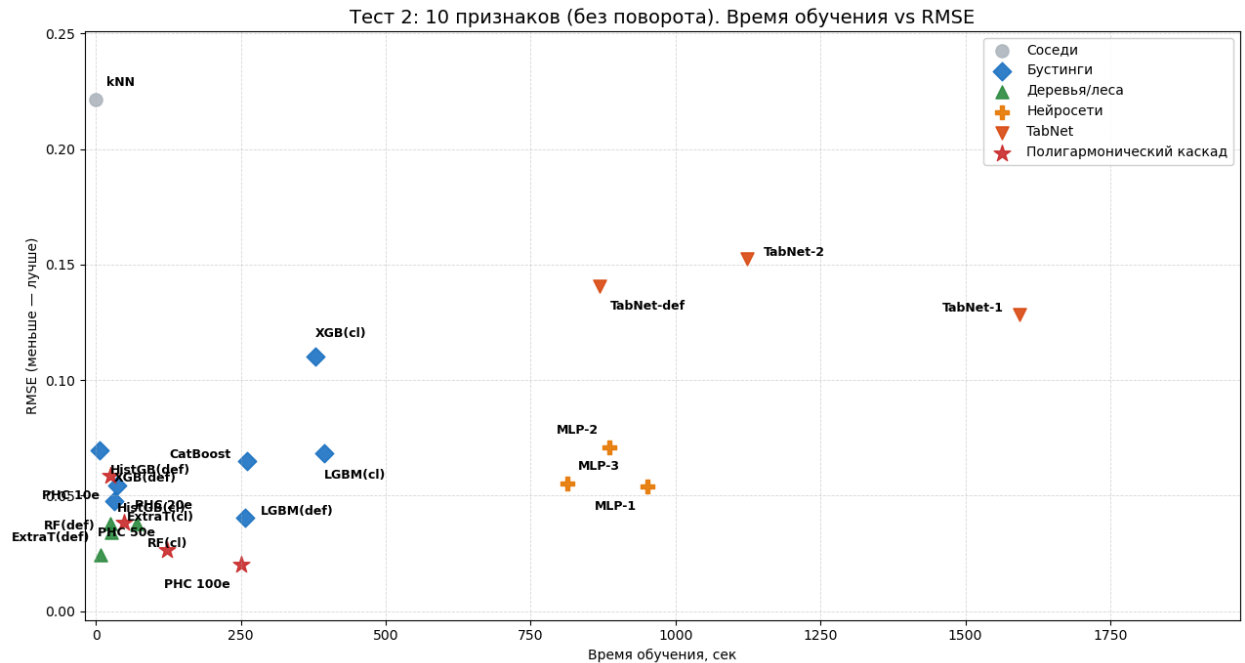
Случайный лес, ExtraTrees и k-NN показали в первом раунде практически идеальные результаты. RMSE в районе 0.002 - 0.005. Впрочем, есть подозрение, что в первом раунде для этих методов (для k-NN точно) случилось просто запоминание всех точек поверхности (что уже нельзя будет проверить во всех следующих раундах).

Полигармонический каскад на 50-ти эпохах показал результат на уровне лучших бустингов, хотя медленнее большей части из них. Но если дать ему еще чуть больше времени, то на 100 эпохах (около 4-х минут обучения) выходит на показатель RMSE 0.016.

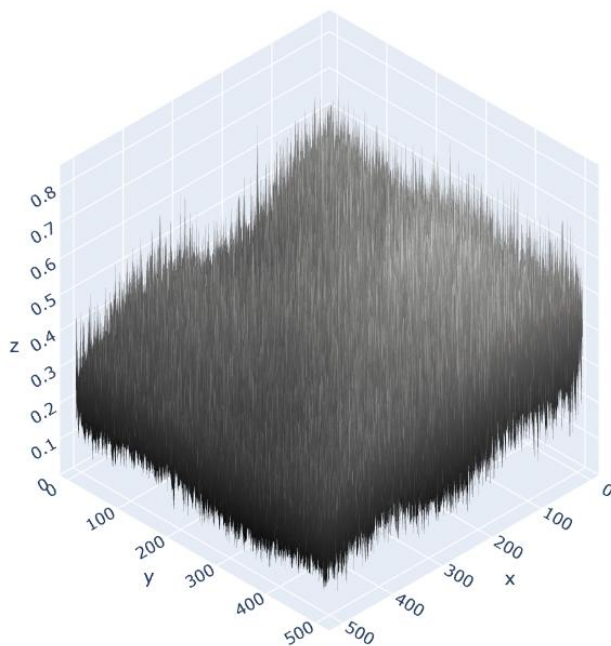
Т.е. полигармонический каскад не лучший на старте (kNN и ExtraTrees в 2D переедут его катком), но он быстро выходит в зону “очень прилично” как аппроксиматор сложной функции.

Разминочный раунд 2. Шум появляется

Правила меняются. Двумерный спринт заканчивается. Алгоритмы получают 10 признаков: 2 полезных и 8 шумовых (которые, напомним, являются перетасованными x и y).



k-NN мгновенно умирает и выбывает из соревнования. Его RMSE прыгает с 0.003 до 0.22. В десятимерном (всего лишь) пространстве понятие “ближайший сосед” теряет смысл. Метод, который только что в предыдущем раунде идеально воспроизвёл картинку, превратился в тыкву. Проклятие размерности во всей красе.



В качестве поверхности на тестовом множестве k-NN получил что-то похожее на шерсть. Красиво, но совсем неправильно.

Деревья больше не выдают идеальный результат, но держатся отлично в диапазоне RMSE 0.0242 – 0.038. Они умеют перебирать признаки и находить полезные. Лучшее из них работает ExtraTrees “из коробки” получая 0.0242 менее чем за 8 секунд.

Бустинги начинают немного потеть, перебираясь в диапазон $RMSE > 0.04$. Кстати, забавный момент: “умная” настройка XGBoost от Клода дала провал RMSE 0.11 за более чем 6 минут обучения, а XGBoost с настройками по умолчанию за 32 секунды выдал 0.047.

Нейросети MLP по времени обучались примерно столько же как в прошлый раз – 13-16 минут. Для MLP-1 добавление шума по непонятным причинам сыграло только на пользу, приведя к резкому улучшению результата по сравнению с прошлым раундом с 0.146 до 0.0538. MLP-2 и MLP-3 получили 0.07 и 0.055. В целом, они уже стали сравнимы по качеству с частью бустингов.

TabNet, обучаясь от 15 минут до получаса получили RMSE 0.12 – 0.15. Возможно им нужны другие настройки/режимы, но в используемом здесь состоянии они выглядят слабее остальных.

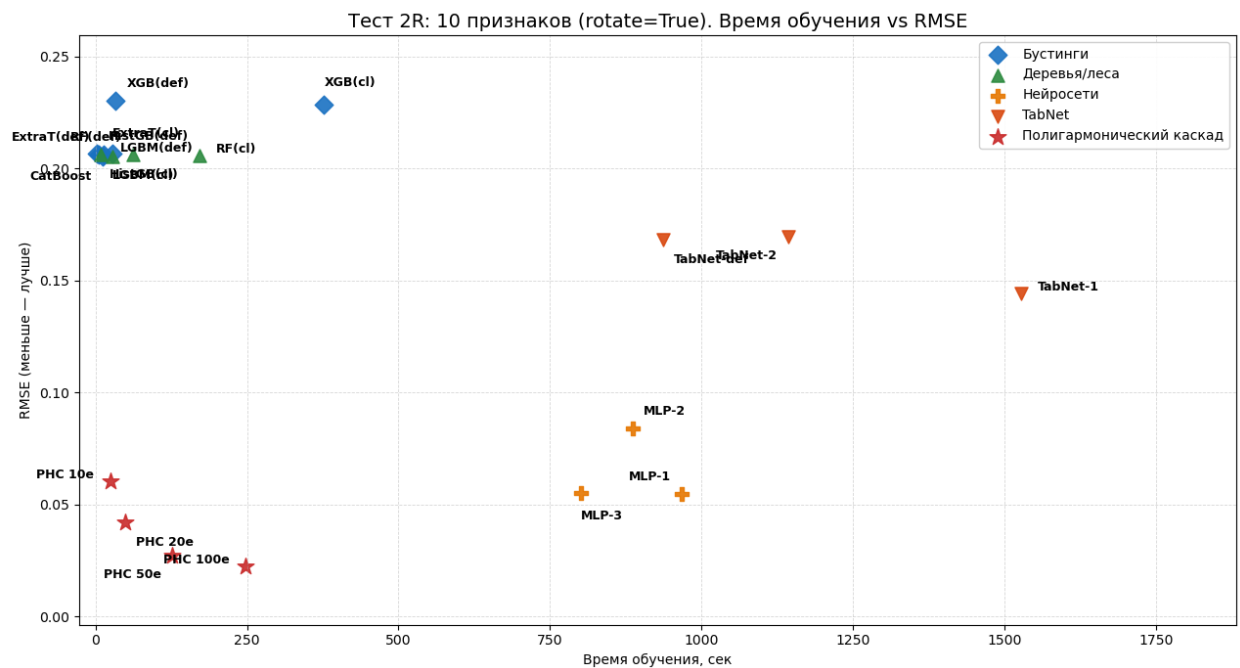
Полигармонический каскад ведет себя очень похожим образом, как и в предыдущем раунде (ему, как видим, добавление 8 столбцов с мусором вообще не мешает), показывая RMSE 0.0266 за 50 эпох (2 минуты обучения) и 0.02 за 100 эпох (4 минуты обучения).

Дальше будет самое интересное.

Разминочный раунд 3. Удар под дых (матричный поворот)

А теперь я сделал тот самый “матричный поворот” системы координат, после которого у алгоритмов начинается экзистенциальный кризис. Информативные признаки перестали быть отдельными колонками, сигнал размазался по 10 измерениям.

Я конечно ожидал, что алгоритмы начнут ошибаться. Но я не ожидал, что это будет массовое самоубийство.



Смотрите на цифры. Порог “я сдаюсь” у нас на отметке RMSE 0.2.

- LightGBM (любой настройки): 0.206 - 0.206
- XGBoost: 0.228 - 0.230
- CatBoost: 0.205
- Random Forest: 0.205
- HistGradientBoosting: 0.206

Все. Абсолютно все деревья и бустинги рухнули в один момент, и на тесте из десяти признаков с поворотом показали результат на уровне линейной регрессии. В скриптах сработал ранний останов – алгоритмы сами поняли, что им задачу не решить и отказались учиться.

А кто выжил?

Нейросети. Их RMSE остался в пределах 0.055 – 0.084.

TabNet – выжил, но выдал уже свой привычный результат 0.14 – 0.16.

Полигармонический каскад – 0.0274 на 50 эпохах и 0.0224 на 100 эпохах. Каскад просто продолжил строить свою поверхность, как ни в чем не бывало.

Первый водораздел.

Именно здесь, на жалких десяти признаках, соревнование распалось на две лиги.

1. **Лига без поворота.** В этих тестах будут продолжать соревнование деревья и бустинги, которые могут работать только когда информативные оси совпадают с осями координат. Будем следить за ними честно и внимательно. Но мы уже знаем: если пространство повернется, они беспомощны.
2. **Лига с поворотом.** Здесь участвуют нейросети (в том числе и TabNet) и полигармонический каскад. Они будут участвовать в обеих категориях: и с поворотом, и без.

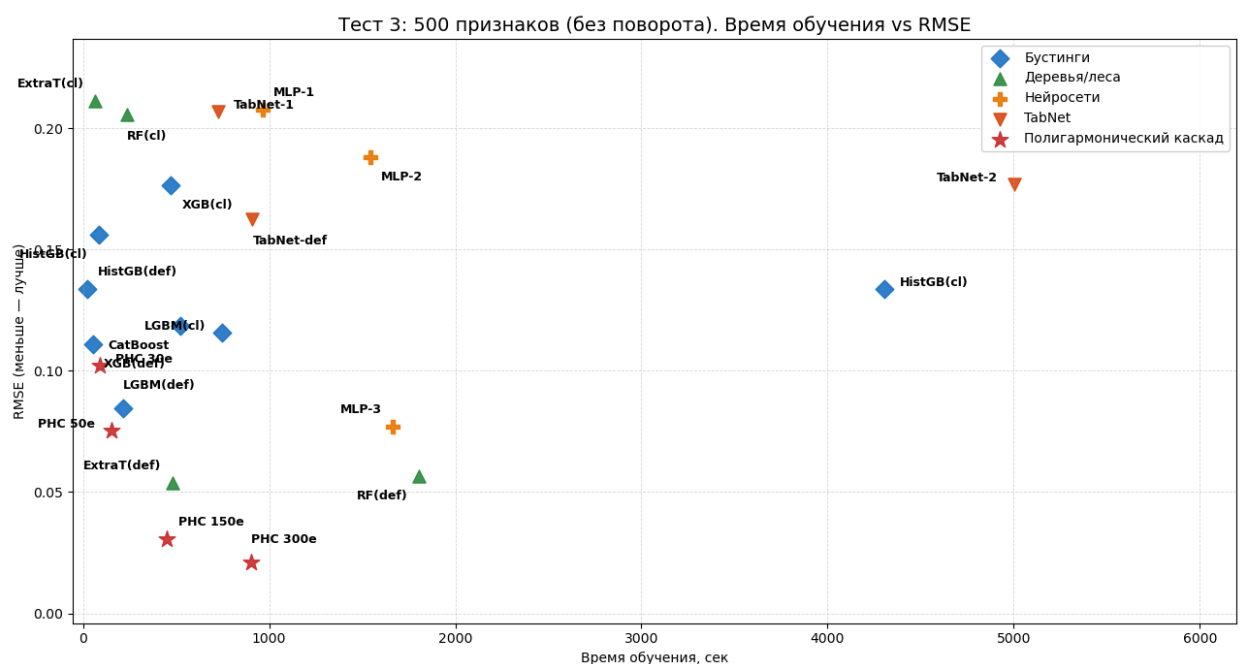
Итак, разминка окончена. Настоящий стресс-тест только начинается.

На первом, полноценном испытании, добавляем 498 признаков-фантомов, идеально имитирующих настоящие. Полезный сигнал теперь составляет 0.4% от входных данных.

Здесь начинается настоящее давление на архитектуру алгоритмов.

Раунд 4. Лес рубят, щепки летят. (500 признаков, без поворота)

Пятьсот признаков это уже серьезно. Но в этом раунде информационные признаки всё еще лежат строго вдоль каких-то двух осей. Теоретически, такие методы как деревья и бустинги способны их найти. Но, что на практике?



Деревья преподносят парадокс. Версии с “тонкой настройкой” от Клода полностью ломаются: RandomForest и ExtraTrees выдают $RMSE > 0.2$ и выбывают из соревнования.

Однако голые версии с настройками по умолчанию показывают себя бойцами: ExtraTrees достигает $RMSE$ 0.0537 за 8 минут, а RandomForest получает $RMSE$ 0.0564 за 30 минут. Что весьма хорошо.

Бустинги.

HistGB с настройками от Claude выдал $RMSE$ 0.156 за полторы минуты. Попытка принудительно докрутить ему количество итераций до огромных значений улучшила результат до 0.134, но стоила 1 час 12 минут.

HistGB (default) обучился очень быстро за 20 секунд до значения $RMSE$ 0.1338 но уперся в тот же невысокий уровень качества как и с настройками от Клода.

LightGBM, XGBoost, CatBoost держатся в районе 0.08 – 0.12. Они вполне работоспособны при пятистах признаках, но уже отстают от деревьев.

Нейросети.

MLP-1 и **TabNet-1** не справились и выбывают.

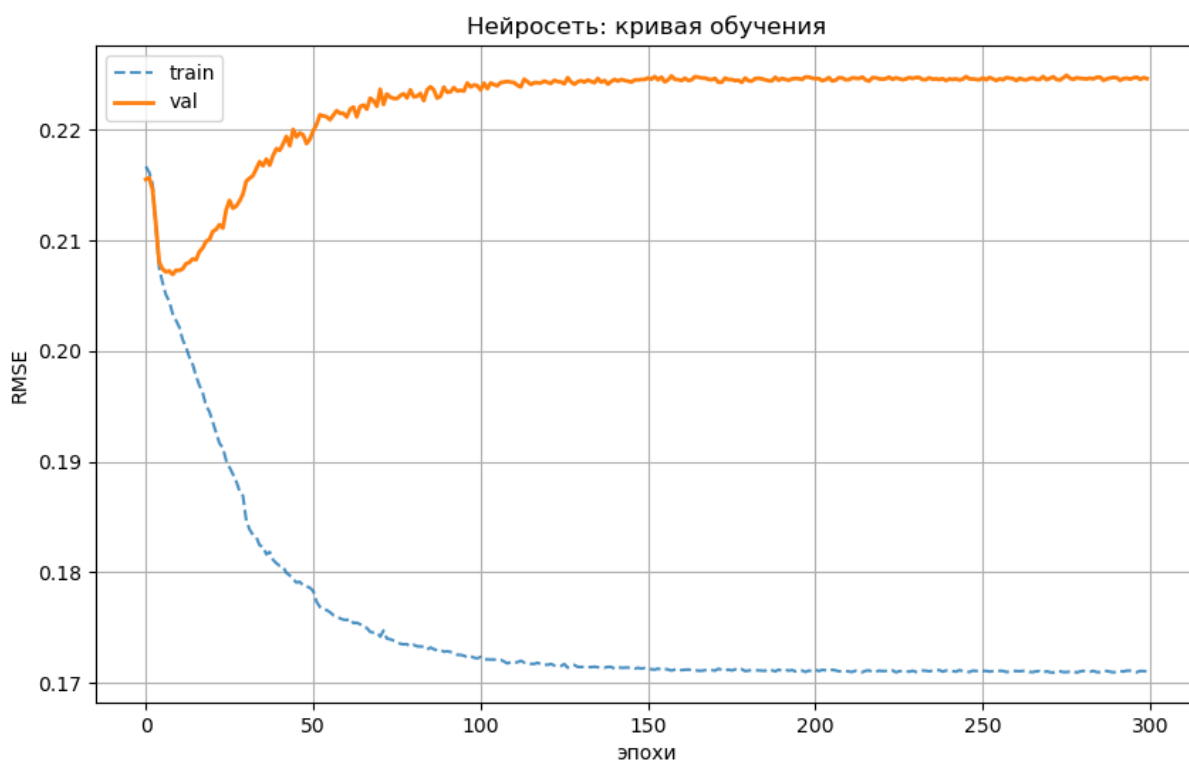


График кривой обучения для MLP-1.

MLP-2 за полчаса обучения достигла показателя RMSE 0.1881, что очень слабый результат. Но все-таки, зачтем его и позволим ей пройти в следующие раунды.

MLP-3 — молодец. Она честно обучилась за полчаса и показала результат RMSE 0.077, что лучше, чем лучший бустинг (хотя хуже, чем деревья). Медленно, но работает.

TabNet-2 обучалась целых 83 минуты и выдала результат RMSE 0.177, что слабо, но дадим ей шансы участвовать дальше.

TabNet-def лучшая из TabNet, обучилась за 15 минут с результатом RMSE 0.1626.

Полигармонический каскад. Значение RMSE 0.03071 за 7 с половиной минут и RMSE 0.02097 за 15 минут. Это не опечатка. При пятистах признаках он показал ту же точность что и при десяти. И при 500 признаках он уходит по RMSE существенно ниже лучших деревьев/бустингов, причем за вполне умеренное время.

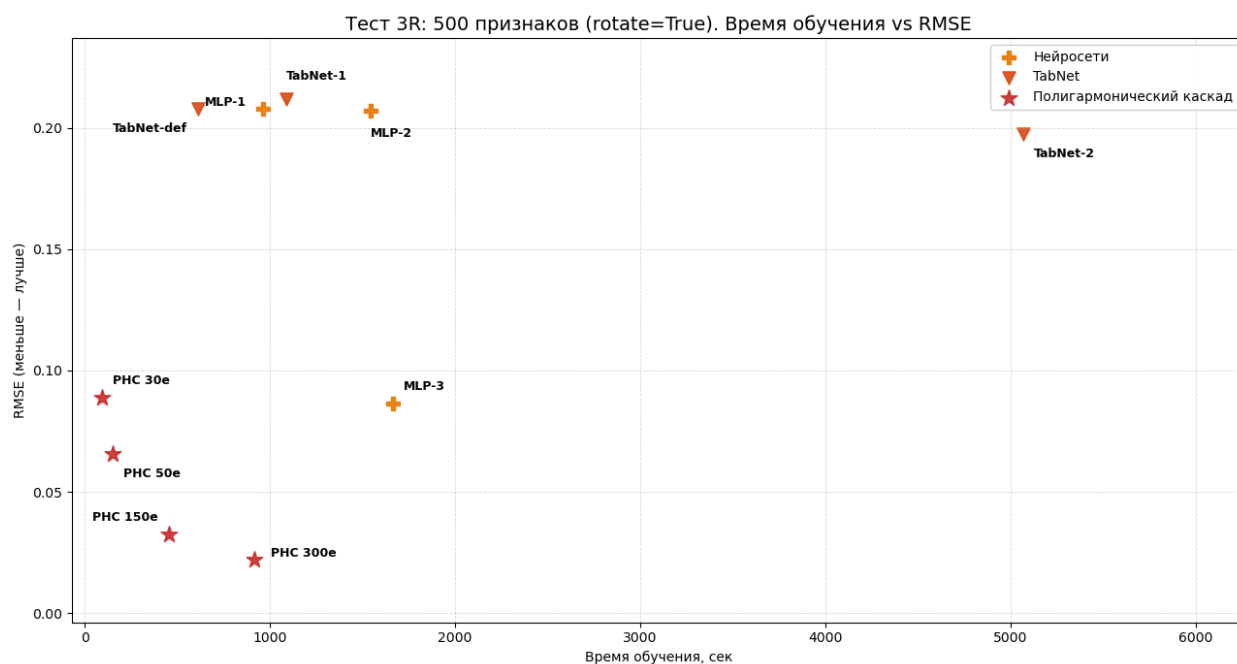
Раунд 5. Мясорубка для нейросетей. (500 признаков, с поворотом)

Включаю режим матричного поворота. Теперь “два полезных столбца” исчезают. Остается только скрытая 2D-плоскость, повернутая в 500-мерном пространстве.

Как уже стало очевидно из разминки (3 раунд), деревья и бустинги после поворота не выживают. Поэтому приводить данные по ним здесь я не буду (хотя, на всякий случай, я их запускал, но сюрпризов не было, они не работоспособны).

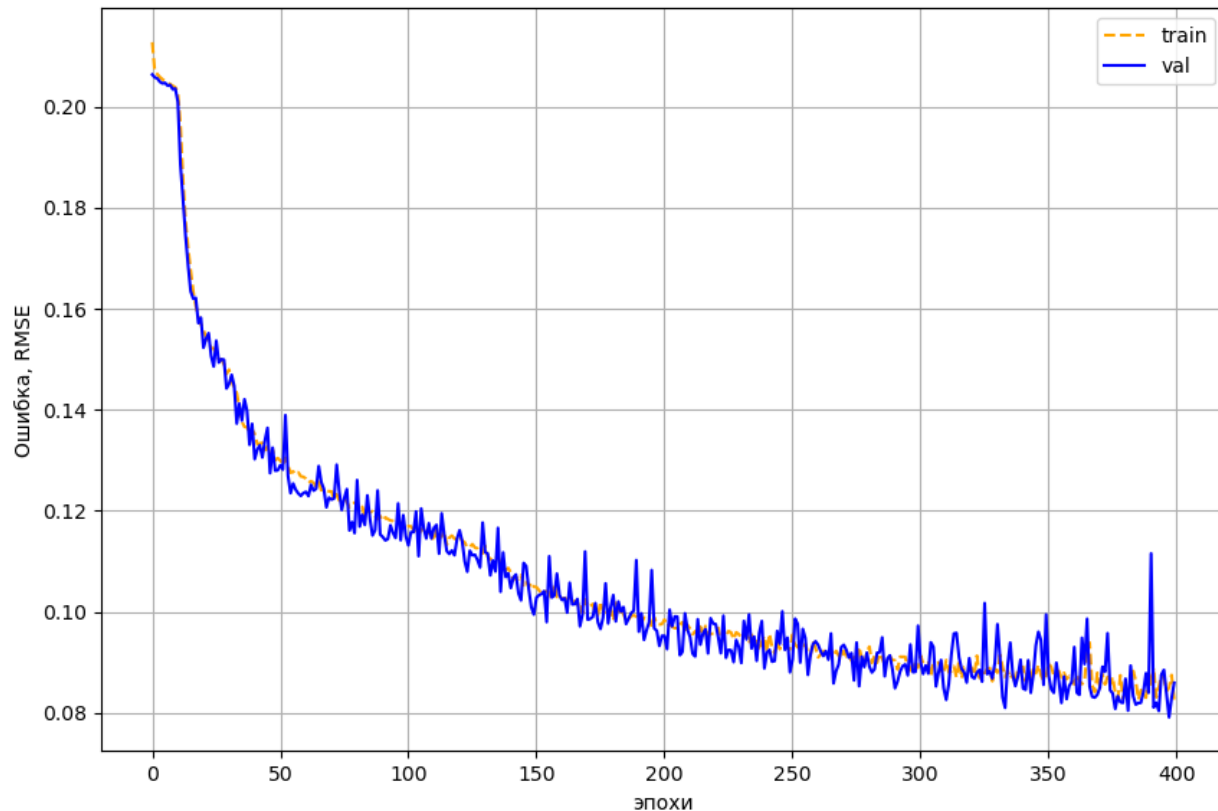
Поле боя очищено. Остались только нейросети и полигармонический каскад.

И тут начинается жестокое.



Слышен хруст ломающихся градиентов и из 6 вариантов нейросетей фактически выживает только одна **MLP-3**. Она потратила 28 минут на обучение и вытянула результат RMSE 0.086. Остальные увы.

Возможно вам будет интересно взглянуть на кривую обучения MLP-3, чем-то похожую на кардиограмму.



Посмотрим далее.

Можно ли считать, что **TabNet-2**, которая смогла через 84 минуты обучения преодолеть вниз отметку RMSE 0.2 и получить 0.1977, справилась? Очевидно, что нет. Но все-таки из-за этой мини-аномалии я решил допустить её к следующему раунду с поворотом.

Полигармонический каскад? Значение RMSE 0.03258 за 7 с половиной минут и RMSE 0.022 за 15 минут.

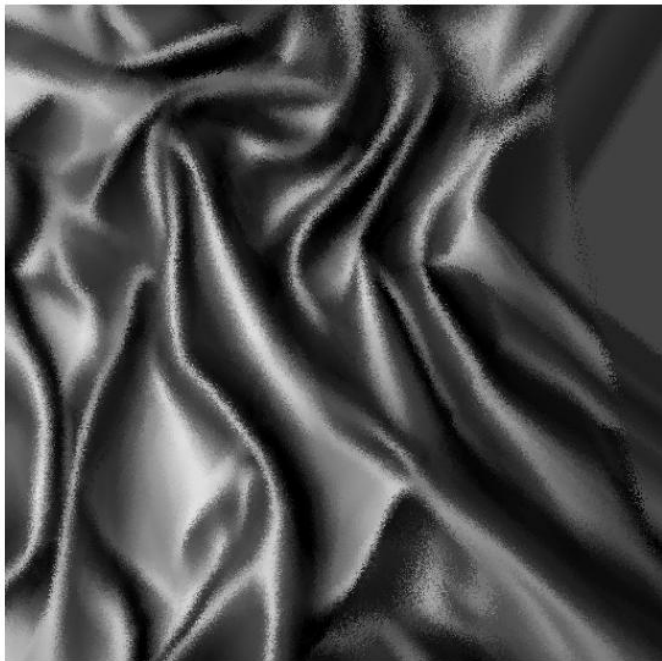
Вы обратили внимание? 0.02097 без поворота из 4-го раунда и 0.022 с поворотом. Разница минимальна. Время обучения то же самое. Каскад поворот практически не заметил.



А вот его график обучения для сравнения. Странной особенностью здесь выглядит то, что кривые ошибки на обучающем множестве и на валидационном, на которые была разбита исходная выборка (напомню, что для итогового результата используется дополнительно сгенерированное еще одна тестовая), практически совпадают и сливаются. Даже намеки на какой-либо процесс переобучения отсутствуют.

Сравним восстановленные изображения по результатам обработки тестового множества нейросетью MLP-3 и полигармоническим каскадом.

Neural Network, $m=500$
RMSE=0.086250



Полигармонический каскад, $m=500$
RMSE=0.022006



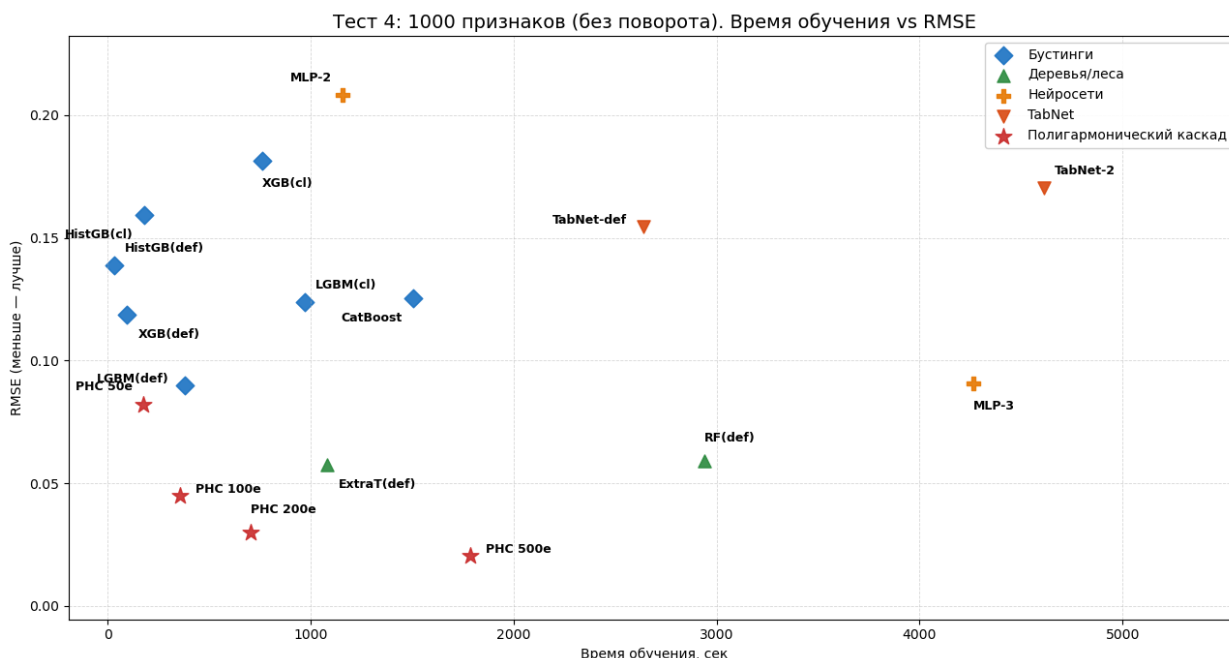
Разницу хорошо видно глазами.

Но 500 признаков, это еще не космос! Посмотрим, что произойдет, когда мы удвоим, а затем учетверим размерность шума.

Итак, у нас теперь 1000 признаков и мы пересекаем психологическую отметку.

Раунд 6. Тысяча признаков без поворота. Воздух становится разреженным

В этом режиме координатные оси не обманывают. Информационные признаки совпадают со столбцами таблицы.



Случайный лес и ExtraTrees в своих конфигурациях “по умолчанию” демонстрируют упрямую живучесть: RMSE 0.057 и 0.059, но ценой 18 и 49 минут вычислений.

Бустинги. Живы и способны находить информационные признаки, но их результаты стагнируют в диапазоне 0.089 – 0.18. Лучший результат из них 0.0899 выдает LightGBM (с дефолтными настройками). XGBoost – 0.118. CatBoost, потратив 25 минут, выдает 0.125. (какие-то методы отработали быстро, какие-то медленнее, но добавление итераций уже не меняло картину)

Нейросети.

MLP-2 на тысяче признаков уже не справилась и покидает соревнование.

MLP-3 провела на дистанции больше часа и показала 0.0906 – чуть хуже, чем при пятистах признаках. Медленно, но качество сопоставимо с лучшим из бустингов.

TabNet-2 и TabNet-def условно живы, но показывают слабые результаты. TabNet-2 – RMSE 0.1703 за 1 час 17 минут. TabNet-def – 0.1546 за 44 минуты.

Полигармонический каскад.

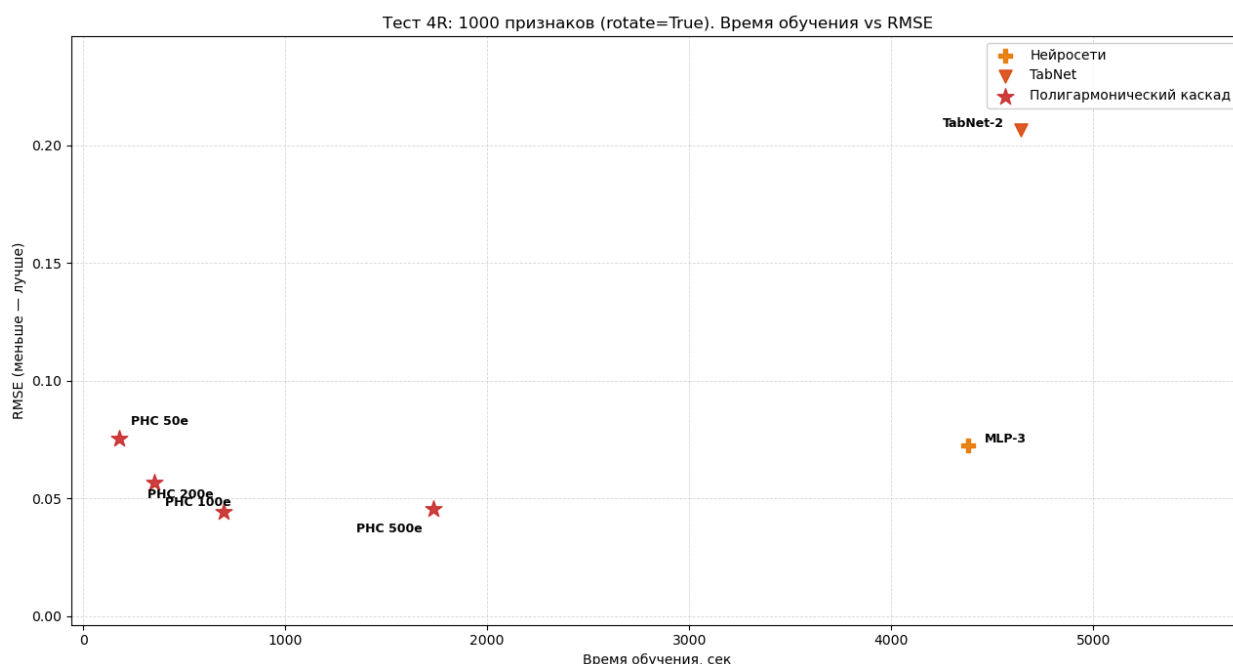
За первые 50 эпох (3 минуты) с RMSE 0.08219 каскад обходит все бустинги и нейросети.

За 100 эпох (6 минут) он выходит на RMSE 0.04492, оставляя позади деревья и становясь лидером.

На двенадцатой минуте результат уже 0.02991. А через полчаса обучения совершенно идеальный 0.02045.

Раунд 7. Тысяча признаков с поворотом. Два одиночества

Я повернул тысячемерное пространство. И их осталось двое.



TabNet-2 окончательно подтвердила неспособность обучиться и выбыла.

Остались только **MLP-3** и **Полигармонический каскад**.

MLP-3 обучалась 1 час 13 минут и вышла на RMSE 0.07225. И это даже почему-то лучше, чем в раунде без поворота.

Полигармонический каскад.

Достиг примерно того же (близкого к MLP3) показателя RMSE 0.07525 за 3 минуты.

Менее чем за 12 минут вышел на уровень 0.04419.

Но затем, на 500-той эпохе показал даже чуть худший результат 0.04549.

Может быть поворот и 1000 признаков в итоге сказались и на качестве каскада?

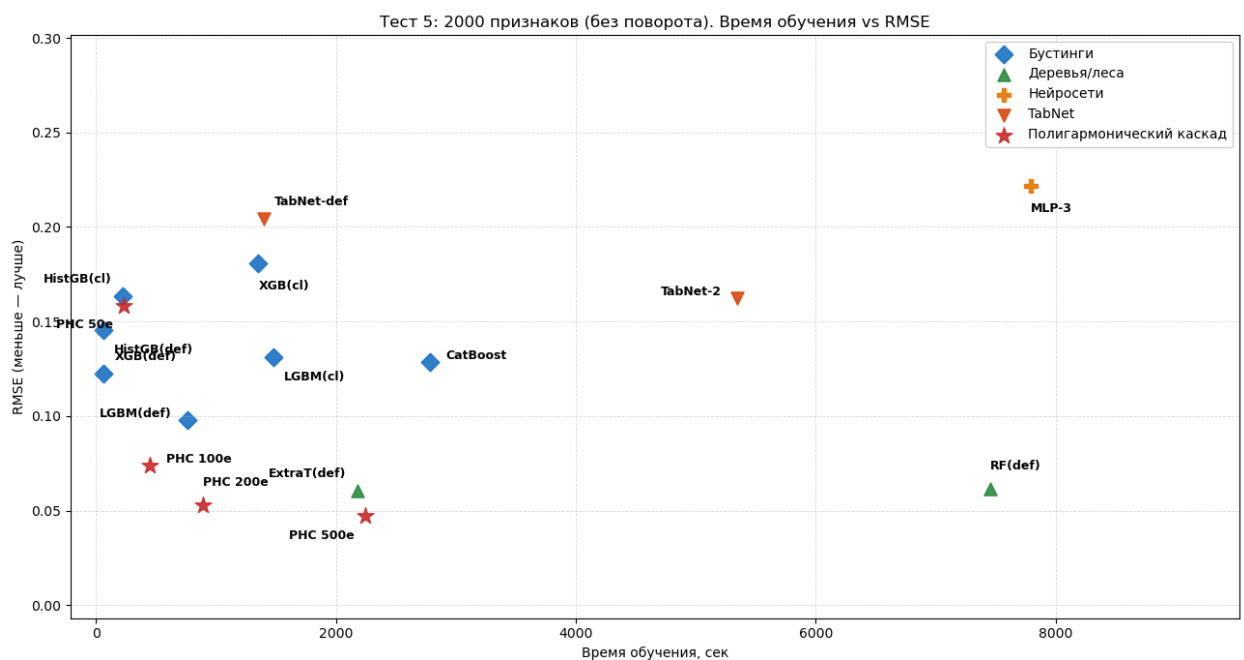
Посмотрим на кривую обучения.



Видимо просто к 500 эпохе неудачно произошел всплеск. Кстати, в отличие от раунда на 500 признаков, здесь уже кривые train и val совсем немного, но уже не полностью совпадают.

Раунд 8. Кода

Две тысячи признаков. Завершающее соревнование в режиме “без поворота”.



Здесь, в заключительном туре сходят с дистанции две нейросети **TabNet-def** и **MLP-3** не сумев справиться с задачей.

Удивительно, но единственной выжившей нейросетью оказалась **TabNet-2**, которая показывала самые слабые результаты весь турнир. Так она его и завершила со своим RMSE 0.1622, достигнув его за полтора часа.

Бустинги.

В режиме “без поворота” выжили и дошли до финиша все.

Но по качеству к 2000 признакам они уже сильно сдали позиции.

Причем бустинги с настройками по умолчанию показали результаты лучше, чем с настройками от Клода.

HistGB(def) и XGB(def) отработали быстро, завершив работу примерно за 1 минуту. Но результаты слабые 0.1453 и 0.1226.

CatBoost думал 46 минут и получил результат 0.1288.

Наилучший RMSE 0.0977 среди бустингов показал LGBM(def), отстрелявшись за 13 минут. Не слишком хорошо, но думаю, это вполне respectable результат, с учетом количества шумовых признаков на входе.

Деревья.

Упорные соперники. В режиме “без поворота” обеспечили наилучшее качество среди всех классических методов.

ExtraTrees обеспечил RMSE 0.06034 за 36 минут.

Случайный лес достиг RMSE 0.0617 за 2 часа 4 минуты.

Не блестяще, но надежно.

И, наконец, **Полигармонический каскад**.

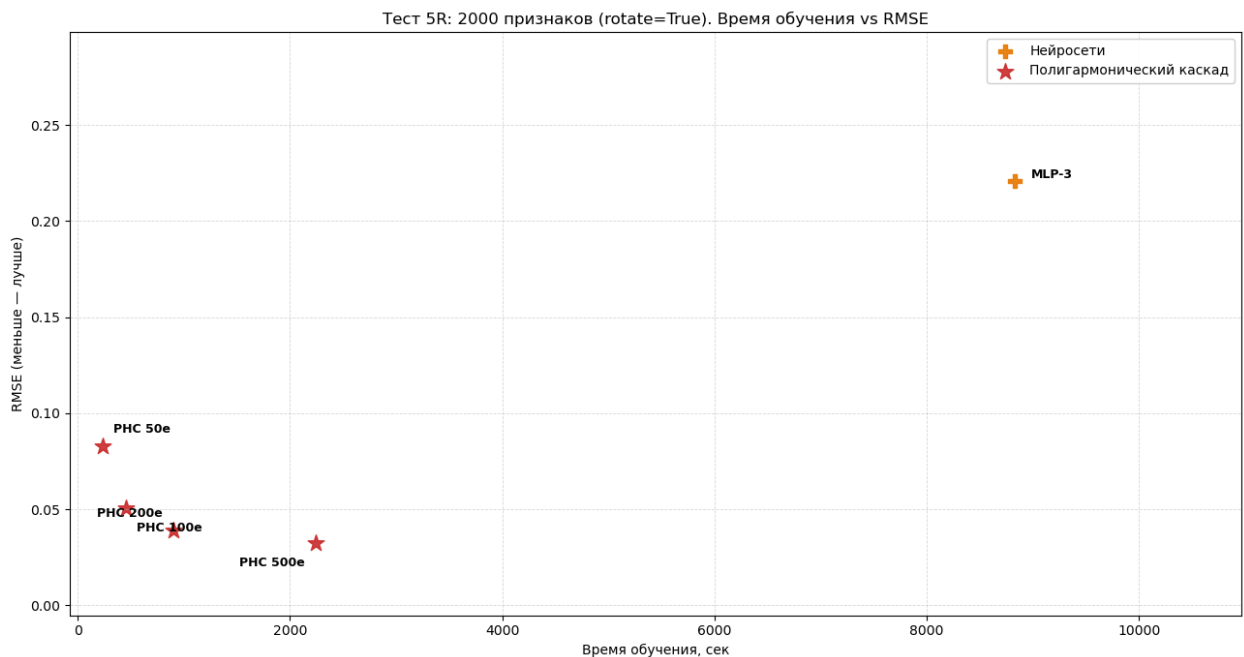
Судя по показателям, 8 раунд дался ему тяжелее предыдущих.

Но тем не менее. Через 15 минут обучения на 200 эпохах он уже имел RMSE 0.05291, что лучше, чем показатели всех остальных участников.

К 500-стам же эпохам (37 минут), он еще улучшил результат до 0.0471.

Раунд 9. Черная кошка в темной комнате

Теперь я повернул двухтысячмерное пространство и запустил финальное испытание.



Сражение закончилось, наступила тишина. Выжил один.

Нейросеть MLP-3, несмотря на то что 1000 признаков была ей по зубам, на 2 тысячах обучиться уже не смогла и выбывает в последнем раунде.

Полигармонический каскад завершает соревнование в гордом одиночестве.

RMSE 0.08254 за четыре минуты.

RMSE 0.0507 за семь с половиной минут.

RMSE 0.03895 за пятнадцать минут.

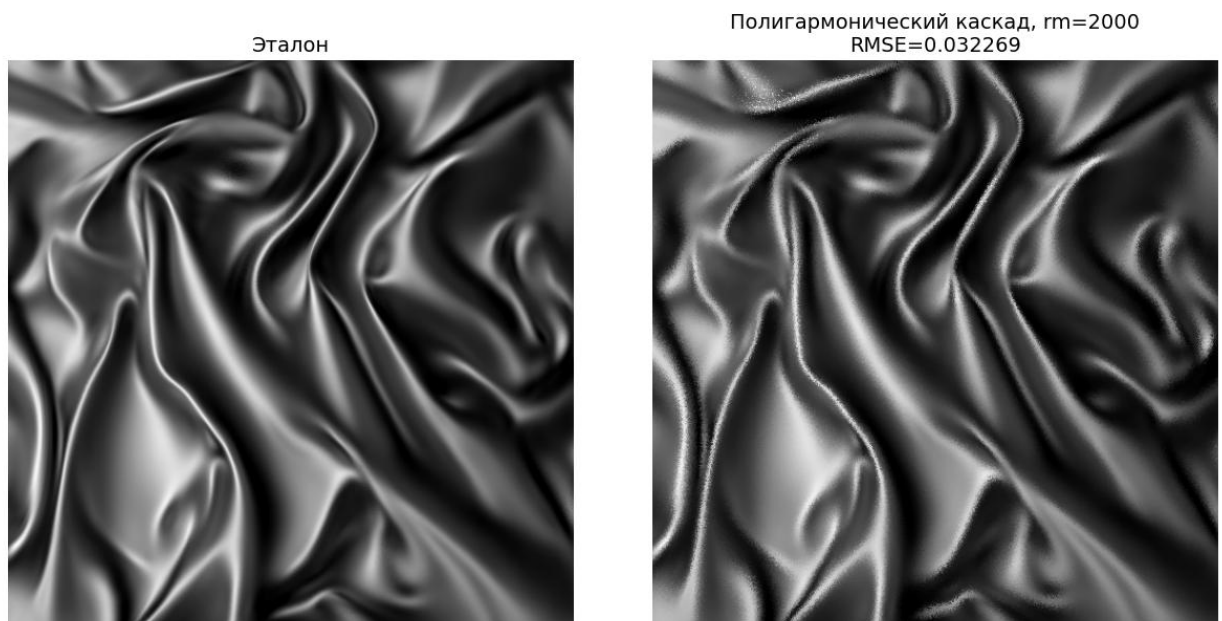
RMSE 0.03227 за 37 минут.

По каким-то непонятным причинам Полигармонический каскад в последнем, самом сложном раунде стал работать еще лучше, чем прежде. Здесь происходит нечто красивое. Он показал результаты не только лучше, чем свои собственные в 8-ом раунде (2 тыс. признаков без поворота). Но и обошел свои же показатели из 7-го раунда, где были 1000 признаков с поворотом.

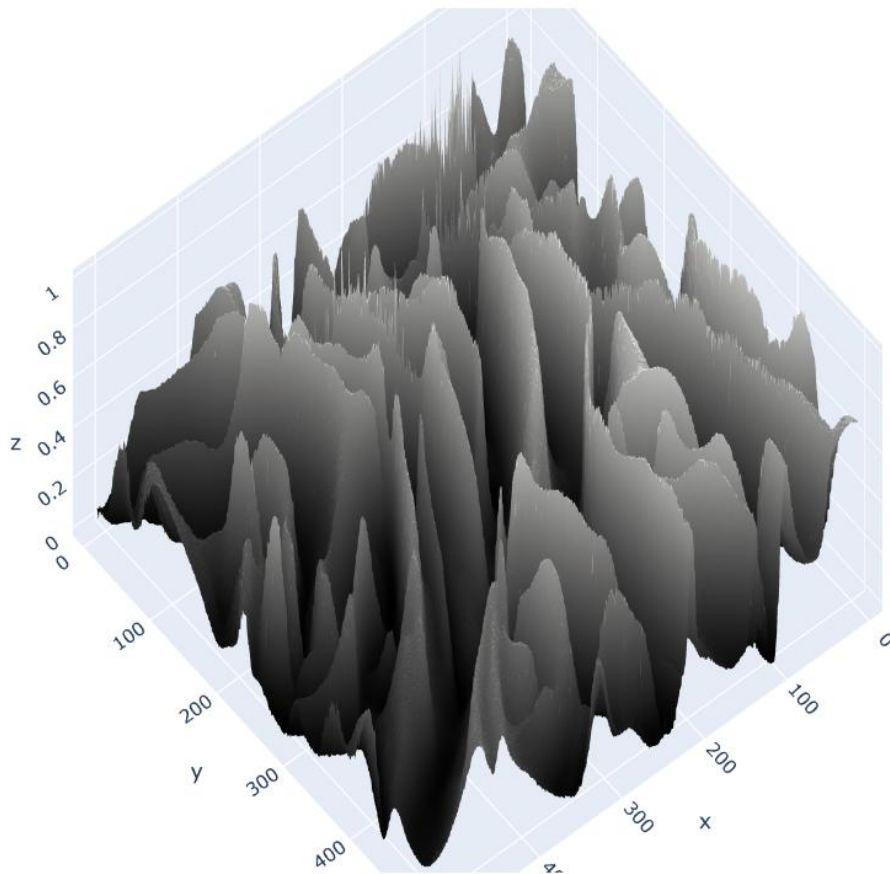
Посмотрим более подробно на результаты работы Полигармонического каскада в финальном раунде.



Так выглядит кривая обучения.



Визуальное сравнение восстановленного изображения с эталонным по тестовой выборке.



Восстановленная по тестовой выборке поверхность.

Послевкусие

Итак, турнир окончен. И что в итоге?

Мы видим, что современный табличный ML, - это гениальный, быстрый, но осе-зависимый механизм. XGBoost, LightGBM, CatBoost, HistGBM, Random Forest, ExtraTrees – все они мыслят категориями “раздели по признаку X, потом по признаку Y”. Пока природа данных выровнена по осям координат, они разрывают всех и вся. Но стоит применить к данным случайное линейное преобразование (которое в реальном мире сплошь и рядом) как эта мощнейшая артиллерия начинает палить вхолостую.

Мы видим, что нейросети обладают правильной интуицией (они понимают поворот). Но обучаются медленнее и при слишком большом количестве шумовых признаков. И когда их стало 2000, они тонут в шуме. Хотя я вполне допускаю, что используемые здесь мной нейросети могли быть реализованы или настроены не самым лучшим возможным образом. Поэтому вопрос об их способностях остается приоткрытым.

И мы увидели новичка. Полигармонический каскад не использует градиентный спуск в классическом виде и не перебирает признаки. В этом тесте он доказал, что способен выхватывать сложную геометрию там, где остальные видят только шум. Ссылки на публикации, описывающие теорию полигармонического каскада, результаты его тестов на других, более официальных бенчмарках приведены в конце статьи.

Небольшое отступление:

Я, конечно, предвижу один из очевидных аргументов в будущих комментариях, которые могут следовать за такими тестами: “А ты сделай РСА перед бустингами(или деревьями), умник!”.

И я провел такой эксперимент (код прилагается в репозитории). Взял лучший из бустингов по результатам восьмого раунда LightGBM и попробовал его применить в девятом раунде с поворотом, но используя перед ним метод главных компонент (РСА). И сколько бы я главных компонент не выбирал 3, 5, 10, 50, 2000... LightGBM выдавал стабильное 0.215 и быстро останавливался. Кажется, РСА не только не смог спасти, но и сам умер.

Спасибо всем, кто дочитал до этого места. Надеюсь, это было хотя бы на 10% так же увлекательно, как и мне проводить это соревнование.

Я провел этот эксперимент в одиночестве, на домашнем железе. Но наука не существует в вакууме (если не требуется условиями эксперимента).

Что дальше?

Поэтому, как я и говорил в эпиграфе: *это приглашение к репликации.*

Всё что описано в статье, можно запустить на вашем компьютере. Никаких черных ящиков, никаких закрытых датасетов.

Репозиторий с полным кодом тестов, генератором данных и реализацией каскада:

<https://github.com/xolod7/black-cat>

Полигармонический каскад

Научные статьи цикла (arXiv):

<https://arxiv.org/abs/2512.12731>

<https://arxiv.org/abs/2512.16718>

<https://arxiv.org/abs/2512.17671>

<https://arxiv.org/abs/2512.19524>

Русскоязычные версии статей:

<https://preprints.ru/article/1324>

<https://preprints.ru/article/1415>

<https://preprints.ru/article/1805>

<https://preprints.ru/article/1963>

Остались вопросы? Хотите обсудить? В комментариях, на GitHub, в личной переписке – я открыт.

Давайте разберемся вместе.